

# Reflexión sobre redes

Juan Pajaro

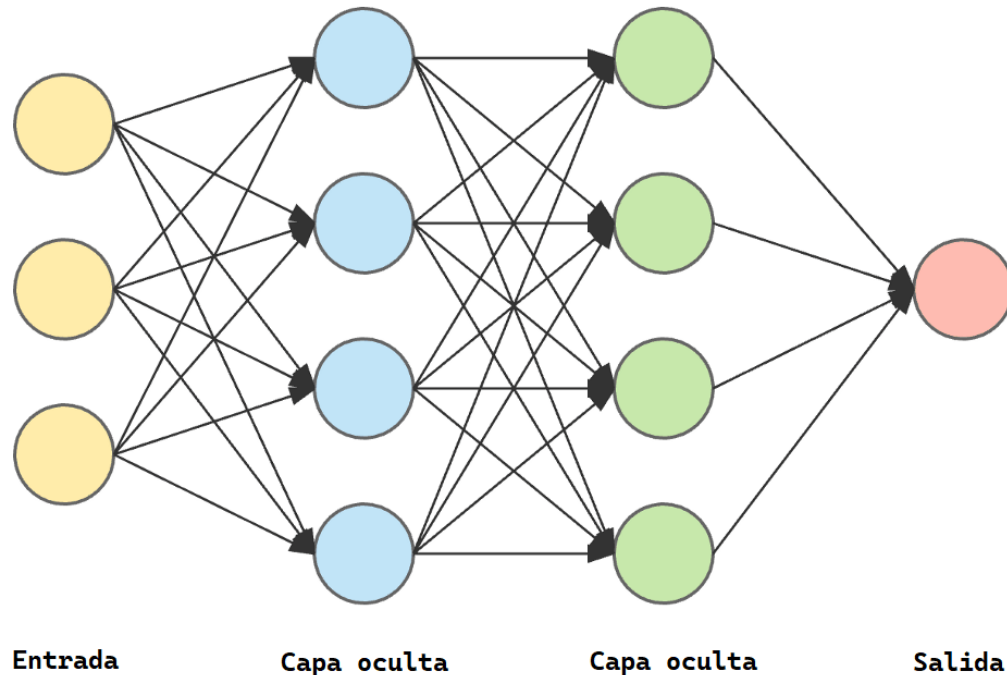
# Tres perspectivas

Lenguaje  
natural

Matemática

Código

# Lenguaje natural



- Existe una composición de capas.
- Cada capa contiene neuronas.
- Las neuronas son funciones lineales y una función de activación.
- Van hacia adelante y hacia atrás.
- Requieren una función de pérdida.
- Una métrica de desempeño.
- Un algoritmo de entrenamiento.
- Trucazo: como ingresan y salen los datos

# Código

```
embedding_dim = 64
model = keras.Sequential(
    [
        layers.Embedding(max_tokens, embedding_dim, name="embedding"),
        layers.GlobalAveragePooling1D(), # Added GlobalAveragePooling1D Layer
        layers.Dense(16, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
```

# Matemática

- Forward propagation is defined as:

- $Z_1 = W_1X + B_1$
- $H = \text{ReLU}(Z_1)$
- $Z_2 = W_2H + B_2$
- $\hat{Y} = \text{softmax}(Z_2)$

- **Backpropagation:** calculate partial derivatives of cost with respect to weights and biases

- $\frac{\partial J_{batch}}{\partial W_1}, \frac{\partial J_{batch}}{\partial W_2}, \frac{\partial J_{batch}}{\partial b_1}, \frac{\partial J_{batch}}{\partial b_2}$

- **Gradient descent:** update weights and biases

- $W_1 := W_1 - \alpha \frac{\partial J_{batch}}{\partial W_1}$
- $W_2 := W_2 - \alpha \frac{\partial J_{batch}}{\partial W_2}$
- $b_1 := b_1 - \alpha \frac{\partial J_{batch}}{\partial b_1}$
- $b_2 := b_2 - \alpha \frac{\partial J_{batch}}{\partial b_2}$

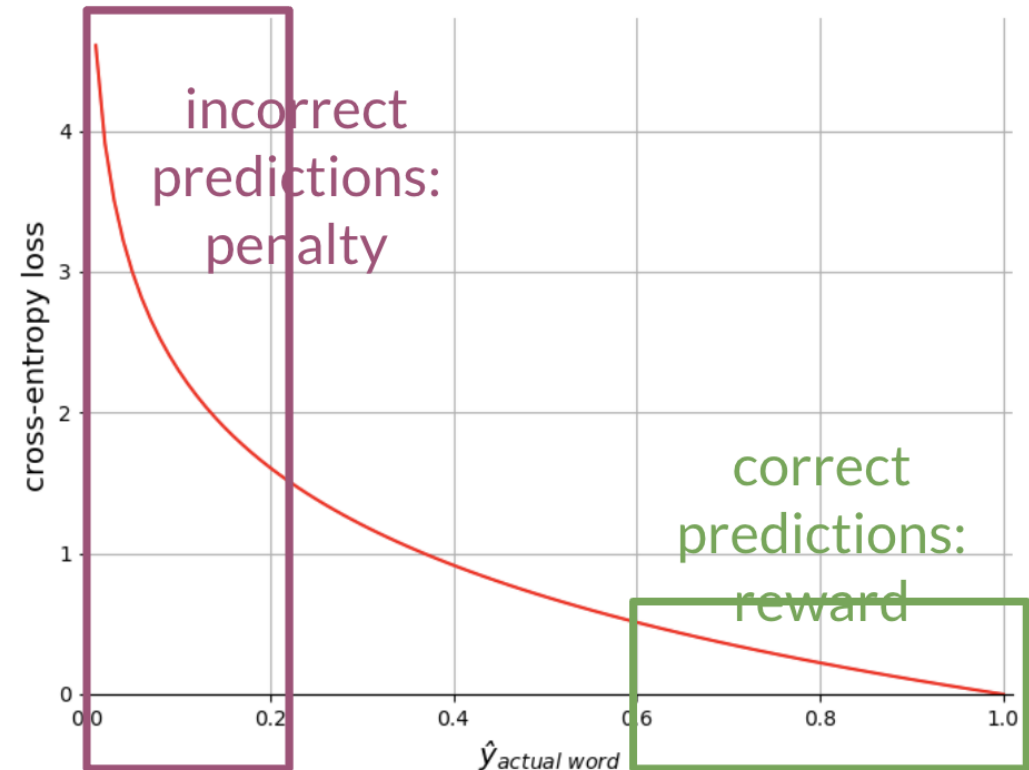
# Matemática

Cross entropy loss:  $J = -\sum_{k=1}^V y_k \log \hat{y}_k$

$$J = -\log \hat{y}_{\text{actual word}}$$

| $y$ |          | $\hat{y}$ |
|-----|----------|-----------|
| 0   | am       | 0.96      |
| 0   | because  | 0.01      |
| 1   | happy    | 0.01      |
| 0   | I        | 0.01      |
| 0   | learning | 0.01      |

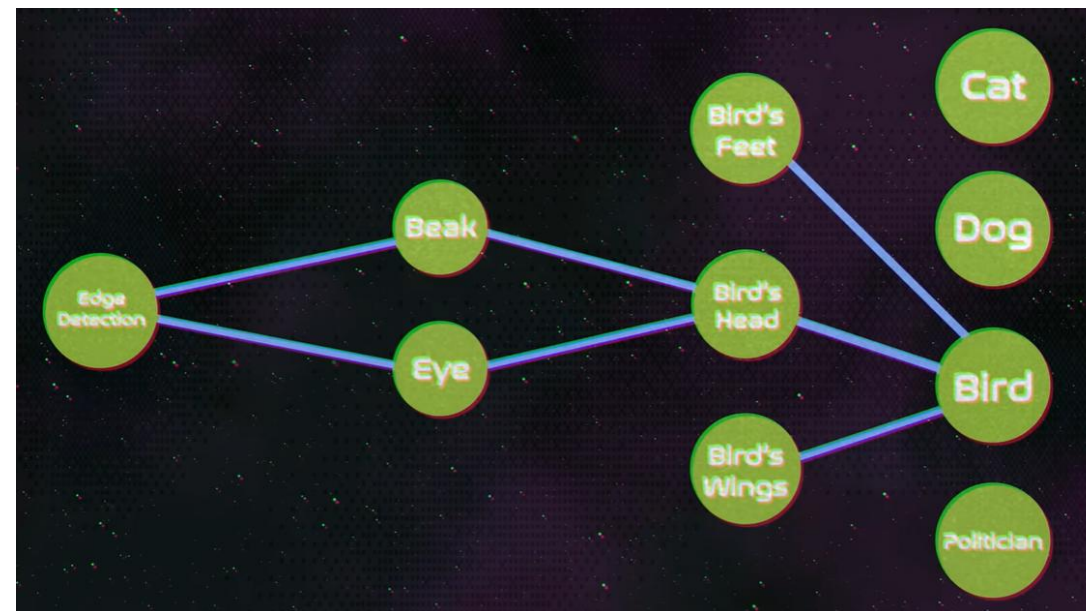
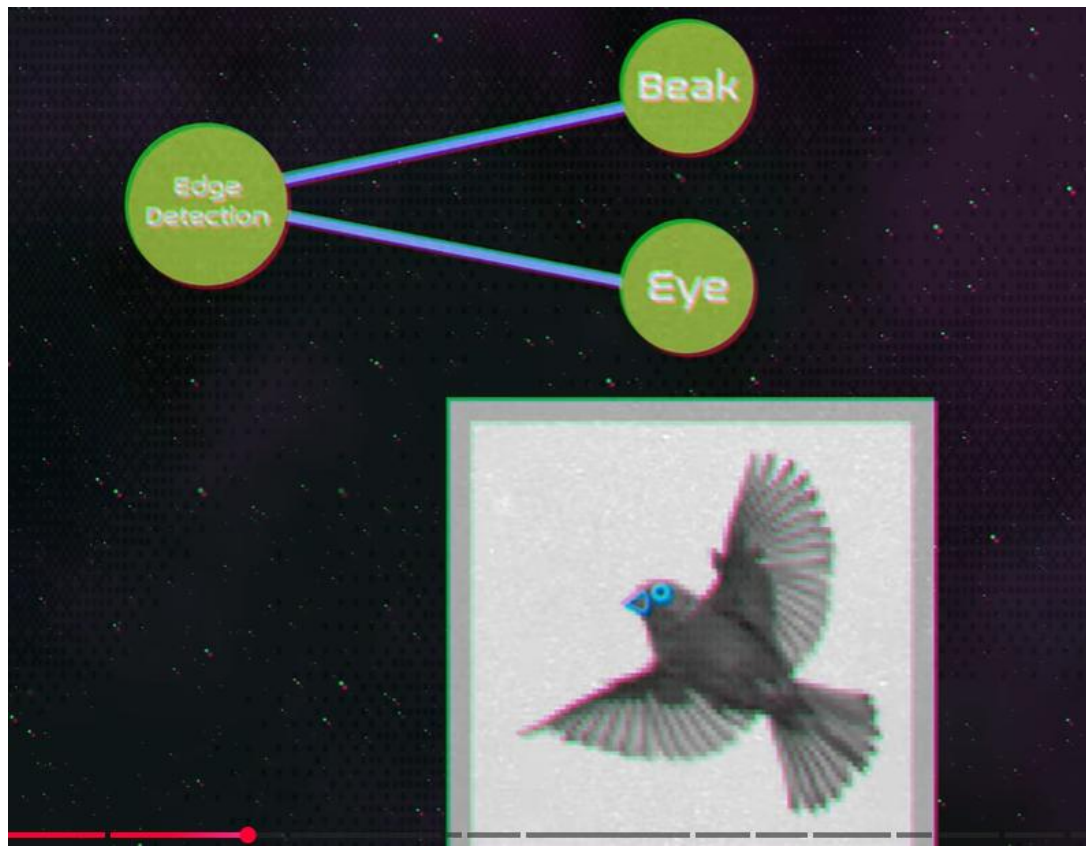
$$\rightarrow J = 4.61$$



# Trucazo

- Capitulo 1 undestanding deep learning

# Trucazo





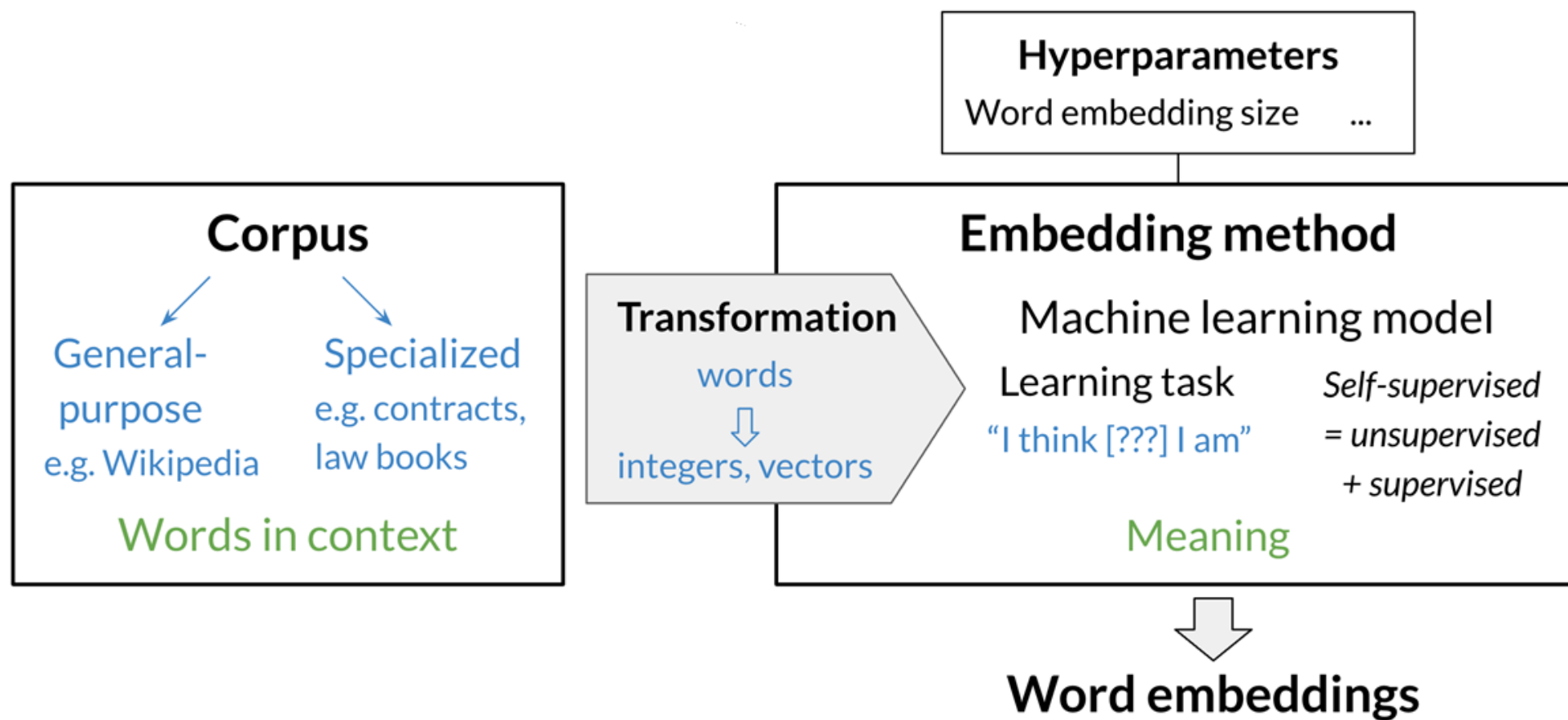
# Trucazo

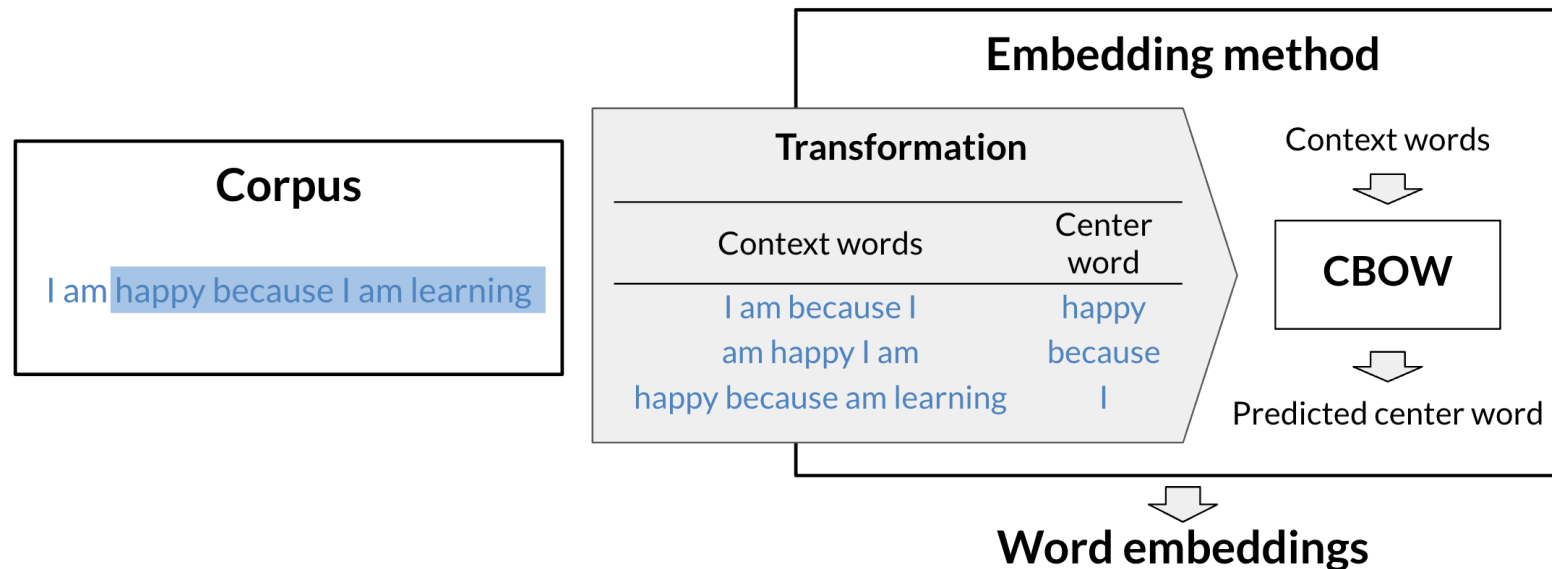
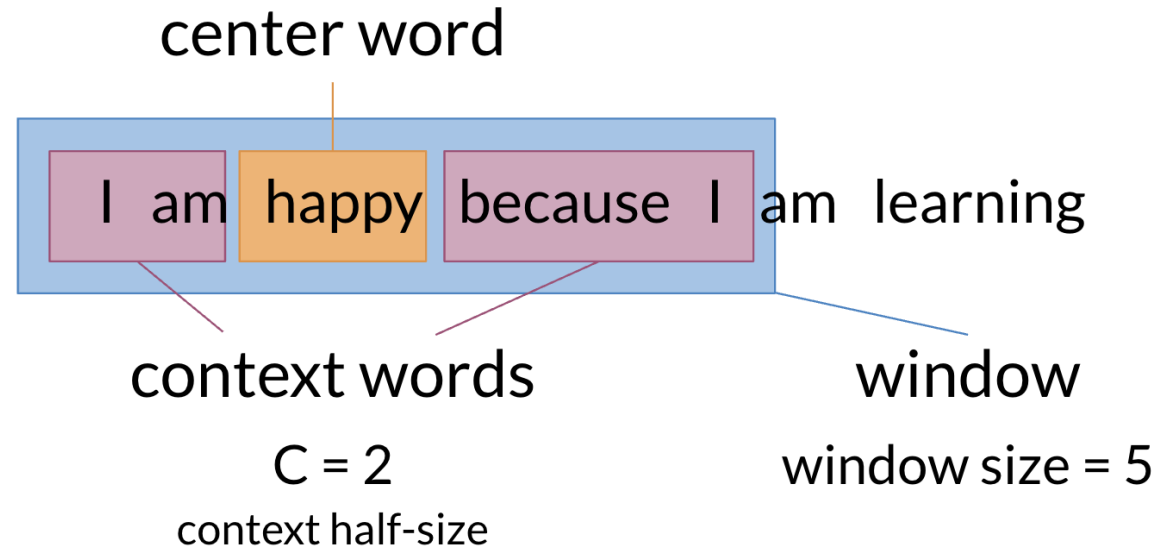
- Ejemplo de nuestro camión
- Libro de deep learning goodfellow
- <https://www.deeplearningbook.org/contents/intro.html>

# Ejemplos

- CITOBOT
- ¿Qué es el cáncer cervical?
- ¿Cómo se detecta?
- Explainable artificial intelligence to diagnose early parkinson's disease



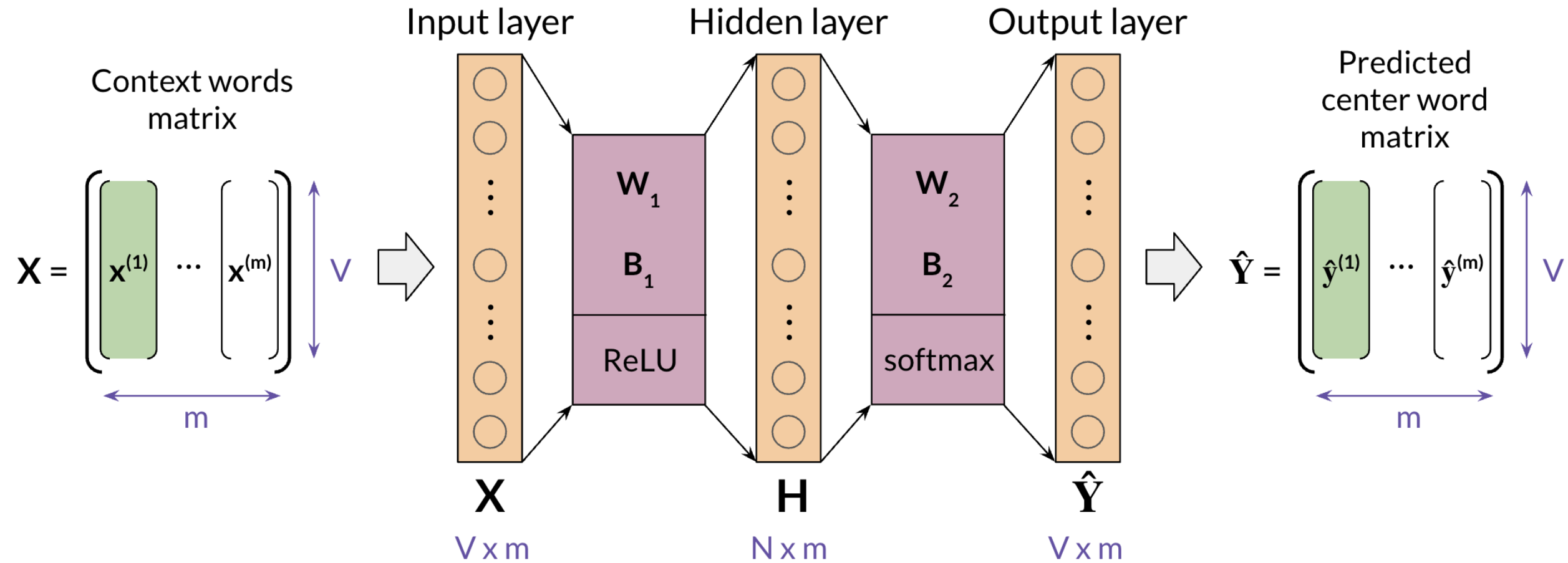




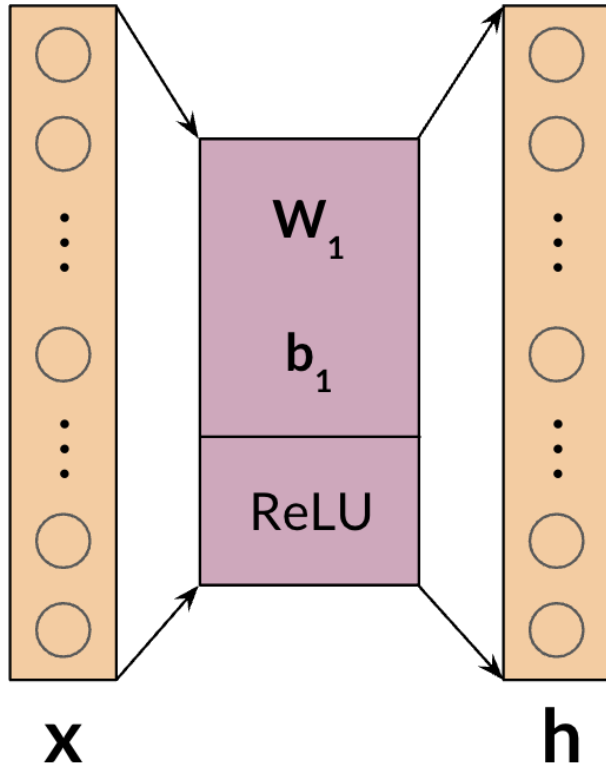
Minable view in the data scientist context

| $x$                              |                             | $y$            |                    |
|----------------------------------|-----------------------------|----------------|--------------------|
| Context words                    | Context words vector        | Center word    | Center word vector |
| <i>I am because I</i>            | [0.25; 0.25; 0; 0.5; 0]     | <i>happy</i>   | [0; 0; 1; 0; 0]    |
| <i>am happy I am</i>             | [0.5; 0; 0.25; 0.25; 0]     | <i>because</i> | [0; 1; 0; 0; 0]    |
| <i>happy because am learning</i> | [0.25; 0.25; 0.25; 0; 0.25] | <i>I</i>       | [0; 0; 0; 1; 0]    |

## "m" training example – one path

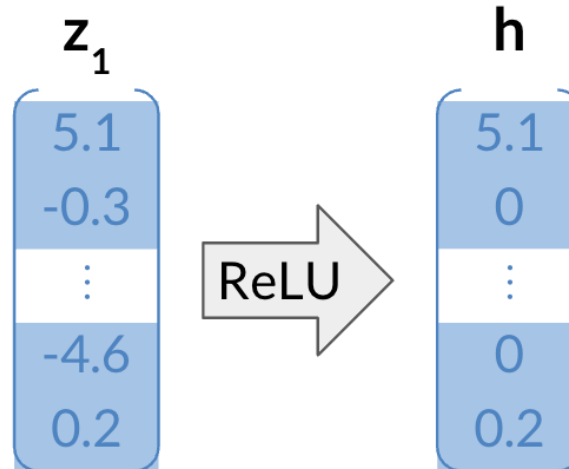


Input layer      Hidden layer

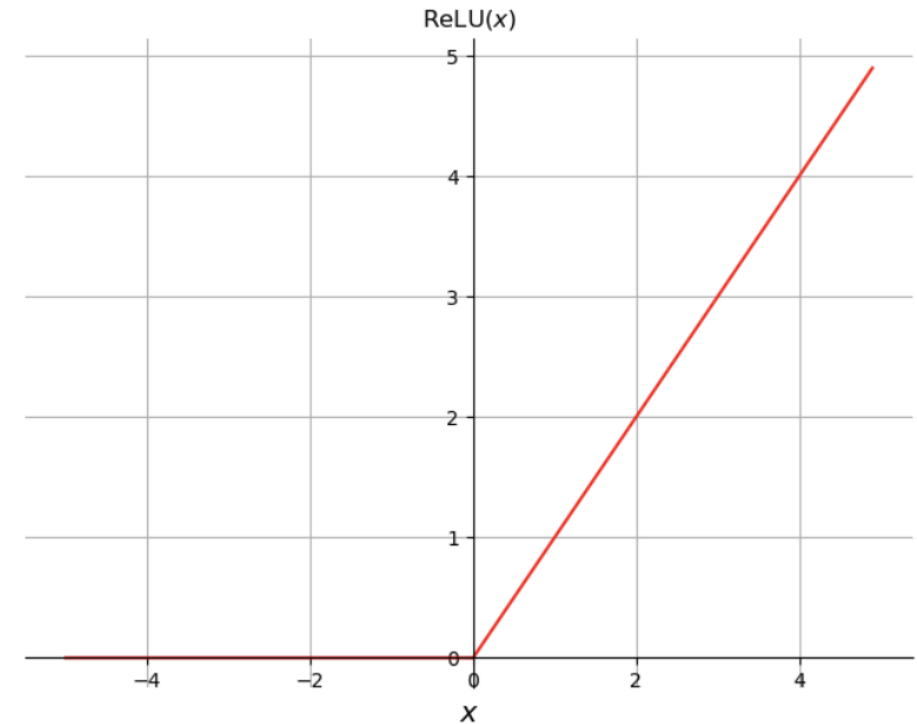


$$z_1 = W_1 x + b_1$$

$$h = \text{ReLU}(z_1)$$

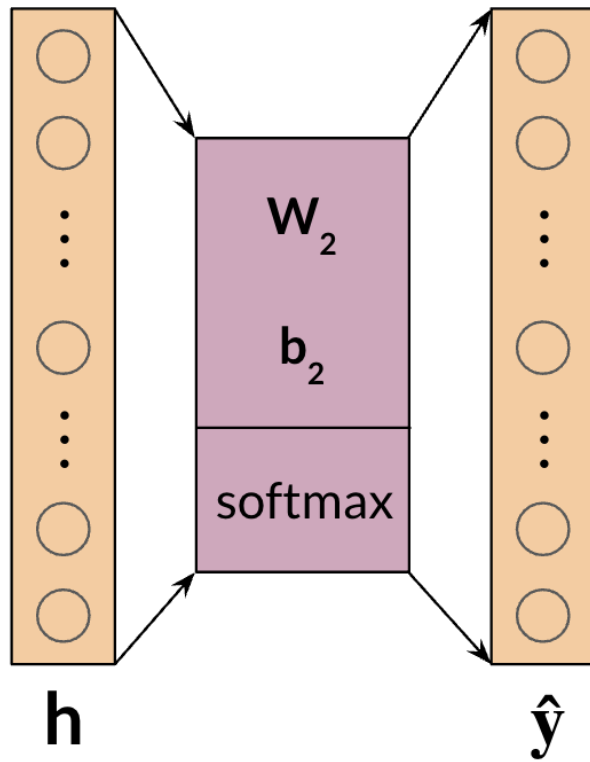


$$\text{ReLU}(x) = \max(0, x)$$



Hidden layer

Output layer



$$\mathbf{z} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

$$\underset{V}{\mathbf{z}} = \begin{bmatrix} z_1 \\ \vdots \\ z_i \\ \vdots \\ z_V \end{bmatrix}$$

softmax

$$\underset{V}{\hat{\mathbf{y}}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_i \\ \vdots \\ \hat{y}_V \end{bmatrix} \begin{matrix} a \\ \vdots \\ \text{happy} \\ \vdots \\ \text{zebra} \end{matrix}$$

Probabilities of  
being center word

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

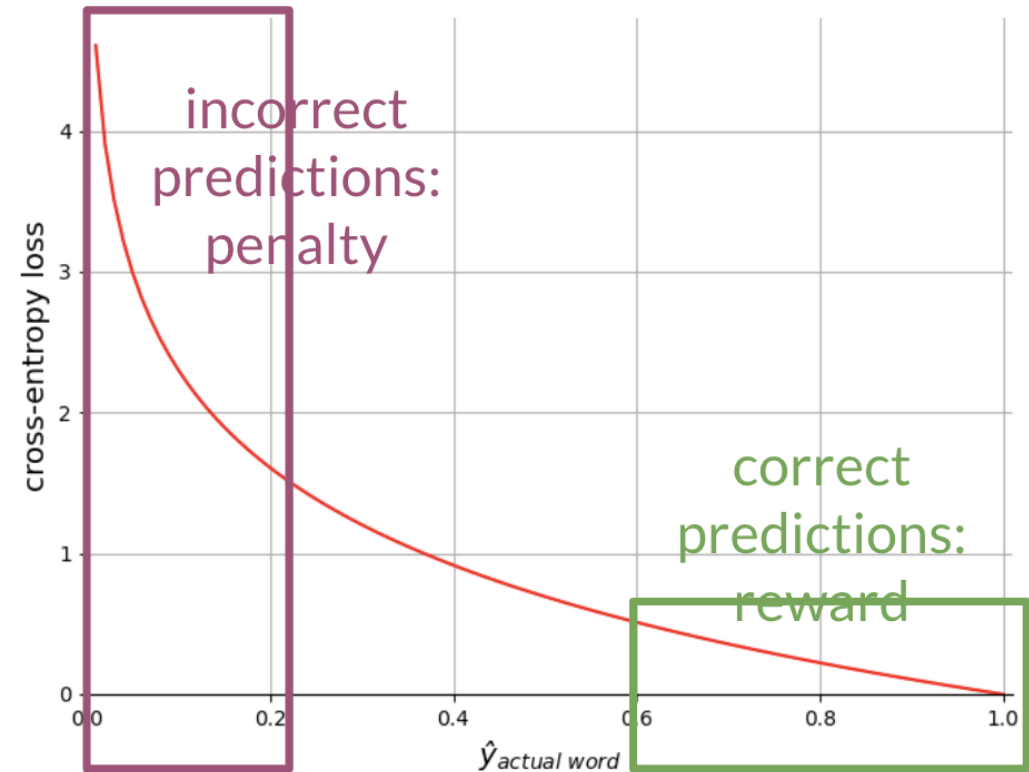


Cross entropy loss: 
$$J = - \sum_{k=1}^V y_k \log \hat{y}_k$$

$J = -\log \hat{y}_{\text{actual word}}$

| $y$ |          | $\hat{y}$ |
|-----|----------|-----------|
| 0   | am       | 0.96      |
| 0   | because  | 0.01      |
| 1   | happy    | 0.01      |
| 0   | I        | 0.01      |
| 0   | learning | 0.01      |

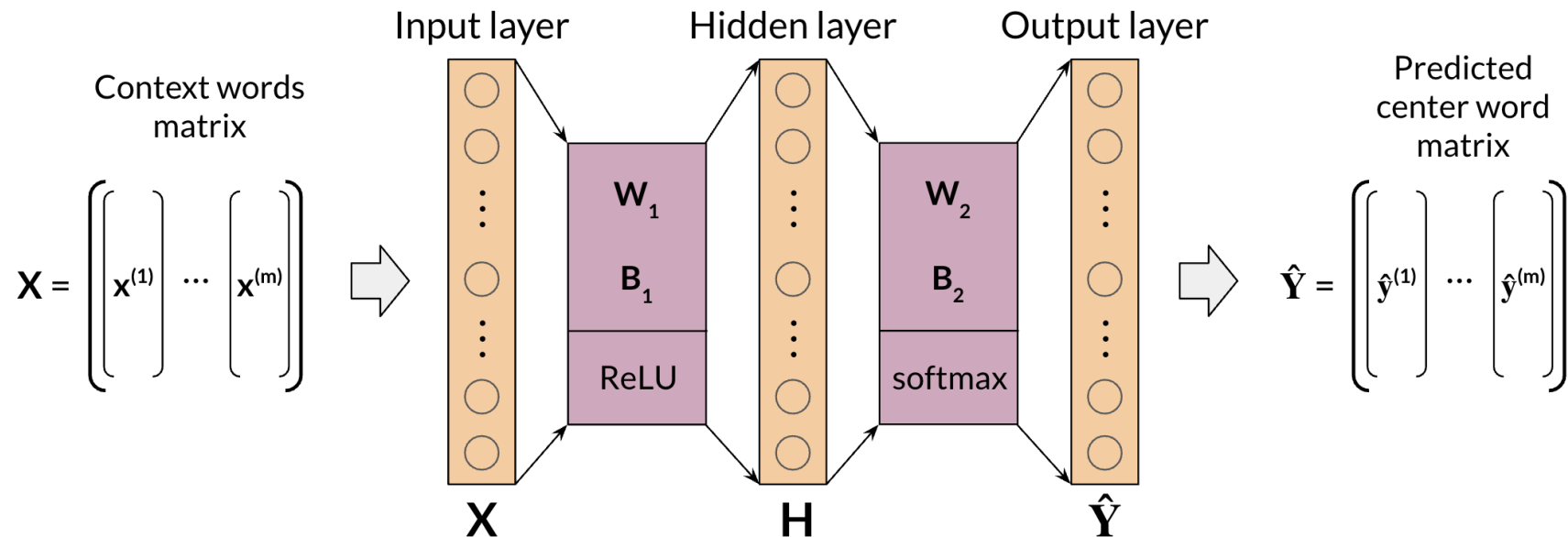
→  $J = 4.61$



Forward propagation is defined as:

- $Z_1 = W_1X + B_1$
- $H = \text{ReLU}(Z_1)$
- $Z_2 = W_2H + B_2$
- $\hat{Y} = \text{softmax}(Z_2)$

$$J_{batch} = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^V y_j^{(i)} \log \hat{y}_j^{(i)}$$



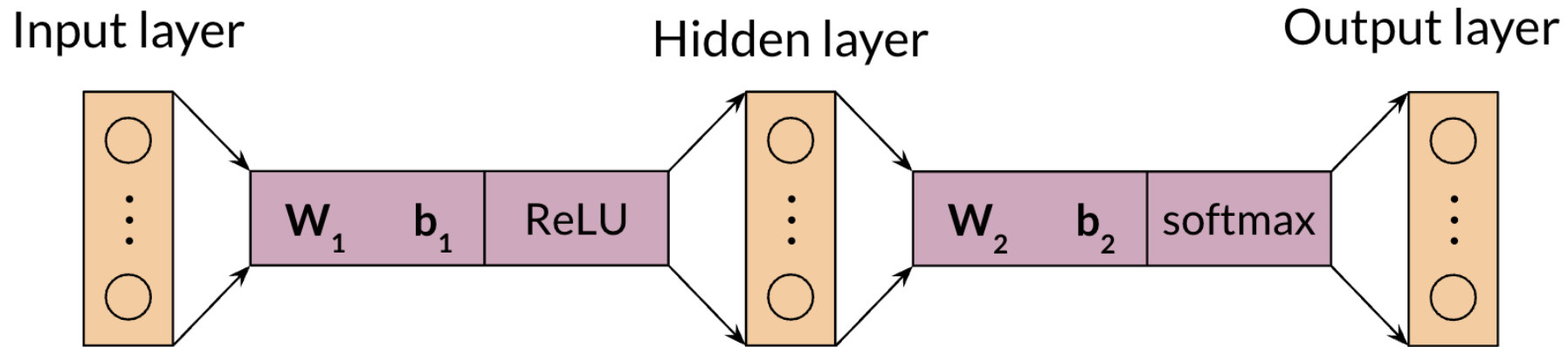
**Backpropagation:** calculate partial derivatives of cost with respect to weights and biases

- $\frac{\partial J_{batch}}{\partial W_1}, \frac{\partial J_{batch}}{\partial W_2}, \frac{\partial J_{batch}}{\partial b_1}, \frac{\partial J_{batch}}{\partial b_2}$

**Gradient descent:** update weights and biases

- $W_1 := W_1 - \alpha \frac{\partial J_{batch}}{\partial W_1}$
- $W_2 := W_2 - \alpha \frac{\partial J_{batch}}{\partial W_2}$
- $b_1 := b_1 - \alpha \frac{\partial J_{batch}}{\partial b_1}$
- $b_2 := b_2 - \alpha \frac{\partial J_{batch}}{\partial b_2}$

## Options 1



$$W_1 = \begin{bmatrix} \text{am} \\ \vdots \\ w^{(1)} & \dots & w^{(V)} \end{bmatrix} \begin{matrix} \updownarrow N \\ \leftarrow V \end{matrix}$$

$$x = \begin{bmatrix} \text{am} \\ \text{because} \\ \text{happy} \\ \text{I} \\ \text{learning} \end{bmatrix} \begin{matrix} \updownarrow V \end{matrix}$$

## Options 2

$$\mathbf{W}_2 = \begin{bmatrix} \boxed{w^{(1)}} \\ \vdots \\ \boxed{w^{(V)}} \end{bmatrix}$$

Diagram illustrating the structure of  $\mathbf{W}_2$ . The matrix is shown as a column vector of word embeddings. The first row contains  $w^{(1)}$  (highlighted in a blue box), followed by an ellipsis, and the last row contains  $w^{(V)}$  (also highlighted in a blue box). A horizontal double-headed arrow below the matrix indicates the width is  $N$ . A vertical double-headed arrow to the right of the matrix indicates the height is  $V$ . The word "am" is written next to the top of the vertical arrow.

$$\mathbf{x} = \begin{bmatrix} \boxed{\text{am}} \\ \text{because} \\ \text{happy} \\ \text{I} \\ \text{learning} \end{bmatrix}$$

Diagram illustrating the structure of  $\mathbf{x}$ . The vector is shown as a column of words. The first element is "am" (highlighted in a blue box), followed by "because", "happy", "I", and "learning". A vertical double-headed arrow to the right of the vector indicates its height is  $V$ .

$$\mathbf{W}_3 = 0.5 (\mathbf{W}_1 + \mathbf{W}_2^T) = \begin{bmatrix} \boxed{\mathbf{w}_3^{(1)}} & \dots & \mathbf{w}_3^{(V)} \end{bmatrix}$$

$\xleftrightarrow{V}$

$\xleftrightarrow{N}$

## Classical methods:

- *Word2vec* (Google, 2013): 1. Continuous bag-of-words (CBOW), 2. Continuous skip-gram.
- *Global Vectors (GloVe)* (Stanford, 2014): factorizes the logarithm of the corpus's word co-occurrence matrix.
- *fastText* (Facebook, 2016): based on the skip-gram model and takes into account the structure of words by representing words as an n-gram of characters.

## Deep learning method, contextual embeddings

- BERT (Google, 2018):
- ELMo (Allen Institute for AI, 2018)
- GPT-2 (OpenAI, 2018)



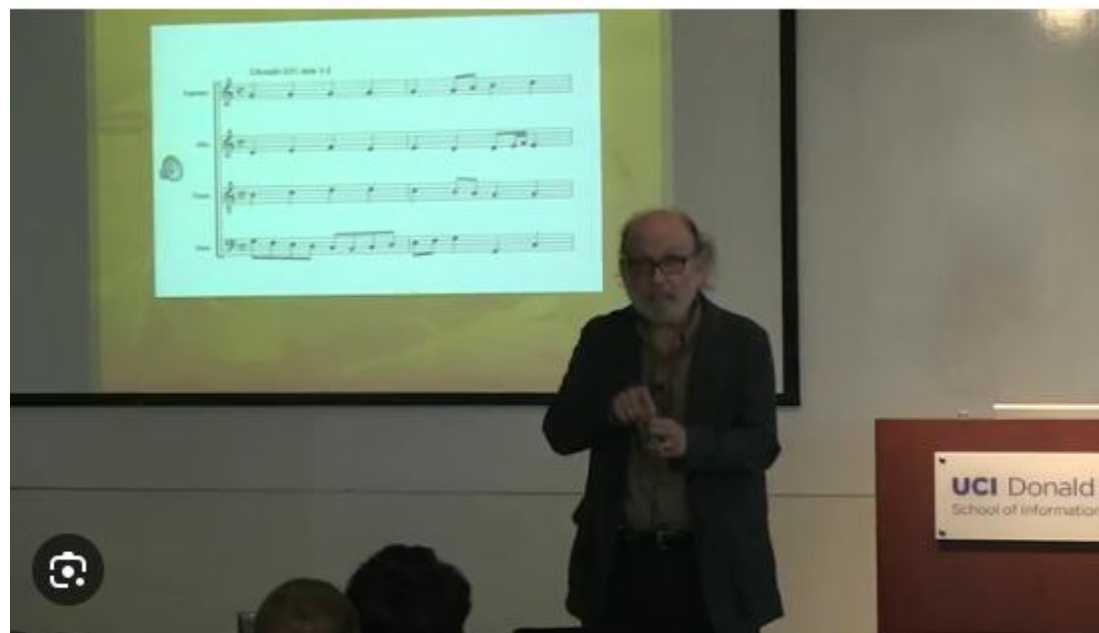
Diapositiva de deeplearning.AI

Curso de sistema RAG



EMI

David Cope



"Computer Synthesis of Musical Creativity" David Cope, UC Santa Cruz

Mirar >